Intro to AI - Final Project

Pathway-finding Visualizer Using A*, Greedy BFS, & Uniform Cost Search Algorithms

Project Group 3 - CRN 74674

Member Names
Abdul Aziz Syed
Areeb Khan
Nehal Rauf

Introduction

The main idea of the project is to model pathfinding as an AI search problem. We wanted the user to be able to build a grid visually, then compare how different search algorithms behave on the exact same setup. The program shows the explored area, the final path, and basic performance numbers

Problem Description

The problem is to move from a chosen start cell to a chosen end cell while avoiding obstacles. The grid is 25 by 25, and movement is limited to up, down, left, and right. Each move has a cost of 1, so the goal is to find a valid path with the lowest total cost

Problem Solution

This is a standard search problem. Every walkable cell is a state. The user chooses the start and goal states.

The available actions are the four basic directions. Walls block movement, and each step costs 1.

That means the best path is the one with the smallest total number of steps

State Space: all valid grid cells

Initial State: start node

Goal State: end node

Actions: up, down, left, right

Constraints: can’t leave grid or pass through walls

Cost function: 1 per move

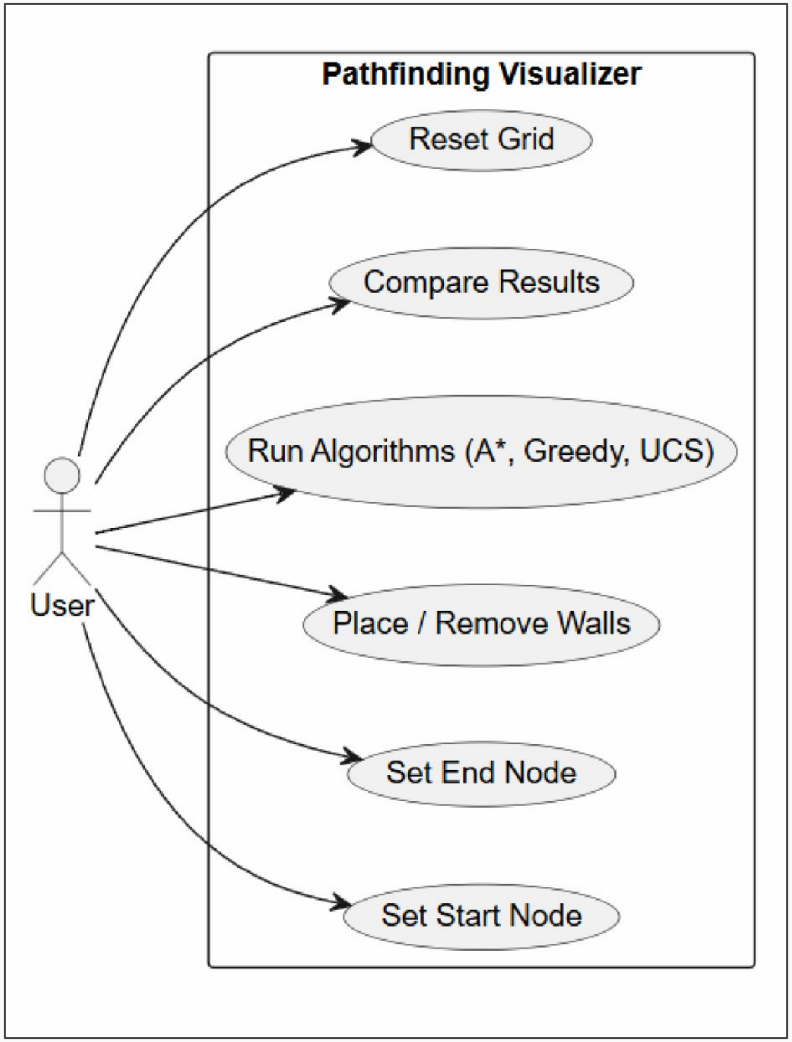
Justification for A.I.

AI is useful here because even on a grid, there can be many possible routes, especially once walls are added. Search algorithms help the system decide what to explore first instead of checking everything randomly. This makes the project a good example of graph search, heuristics, and cost-based decision making.

Use Case Diagram

The user can set the start node, set the end node, place or remove walls, run the algorithms, compare results, and reset the grid.

These use cases directly match the actual buttons and features in the program



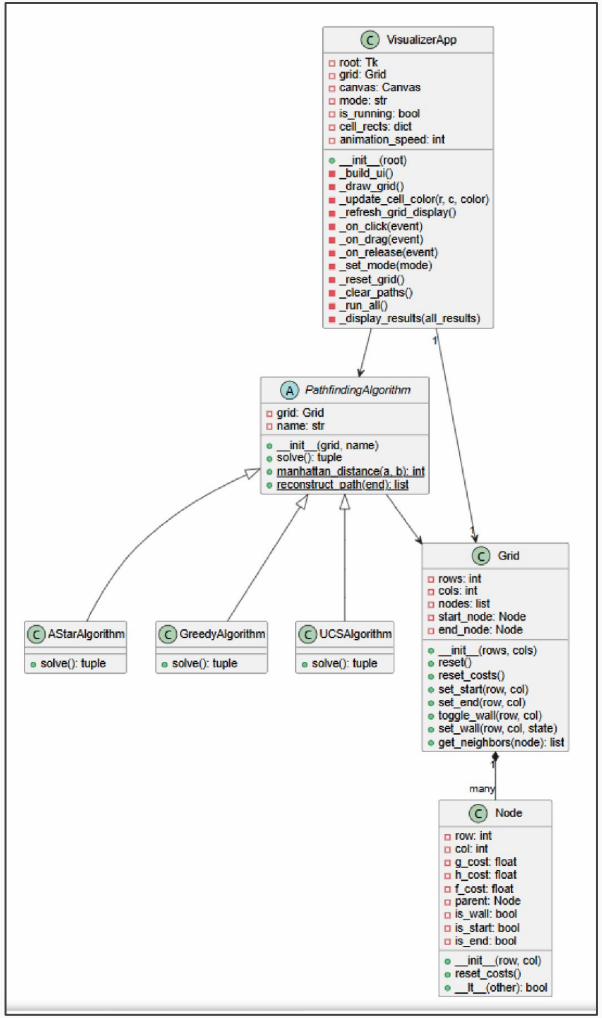
Class Diagram

Node represents each cell.

Grid stores all nodes and manages neighbours, walls, start, and end.

PathfindingAlgorithm is the parent class, and then **A*, Greedy,** and **UCS** each implement their own search logic.

VisualizerApp handles the interface, user input, animation, and result display.”



Code Structure

Node → stores row, col, costs, parent,

wall/start/end flags

Grid → stores nodes and grid operations

PathfindingAlgorithm → shared search functions

AStarAlgorithm, **GreedyAlgorithm**, **UCSAlgorithm**

VisualizerApp → GUI and animation

SOFE3720_FinalProject_Gr3_CRN74764 / PathfindingVisualizer / main.py

Code Blame 658 lines (616 loc) · 26.2 KB Raw

```python
COLOR_PATH_ASTAR = "#2E86C1"
COLOR_PATH_GREEDY = "#F39C12"
COLOR_PATH_UCS = "#8E44AD"
COLOR_GRID_LINE = "#BDC3C7"

class Node: ...
        return self.f_cost < other.f_cost

class Grid: ...
        return neighbors

class PathfindingAlgorithm(ABC): ...
        return path

class AStarAlgorithm(PathfindingAlgorithm): ...
        }

class GreedyAlgorithm(PathfindingAlgorithm): ...
        }

class UCSAlgorithm(PathfindingAlgorithm): ...
        }

class VisualizerApp: ...
                ).grid(row=i + 1, column=j, sticky="nsew")

if __name__ == "__main__":
```

Algorithms/Heuristics

A* Algorithm

- Uses *g+h*
- Balances path cost and heuristic

Greedy Best-First Search Algorithm

- Uses *h* only
- Moves toward goal aggressively

UCS - Uniform Cost Search Algorithm

- Uses *g* only
- Expands lowest actual cost first

Heuristics - Manhattan Distance

Measures how far a node is from the goal using only row and column difference. Agent can only move up, down, left, or right, Manhattan distance is a good fit for this problem for A* & Greedy BFS.

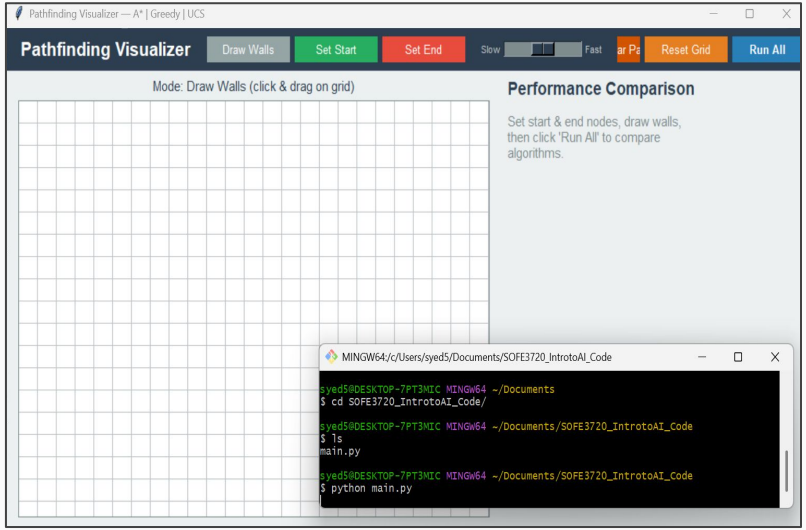
$$|r1 - r2| + |c1 - c2|$$

User Interface (GUI)/Program Features

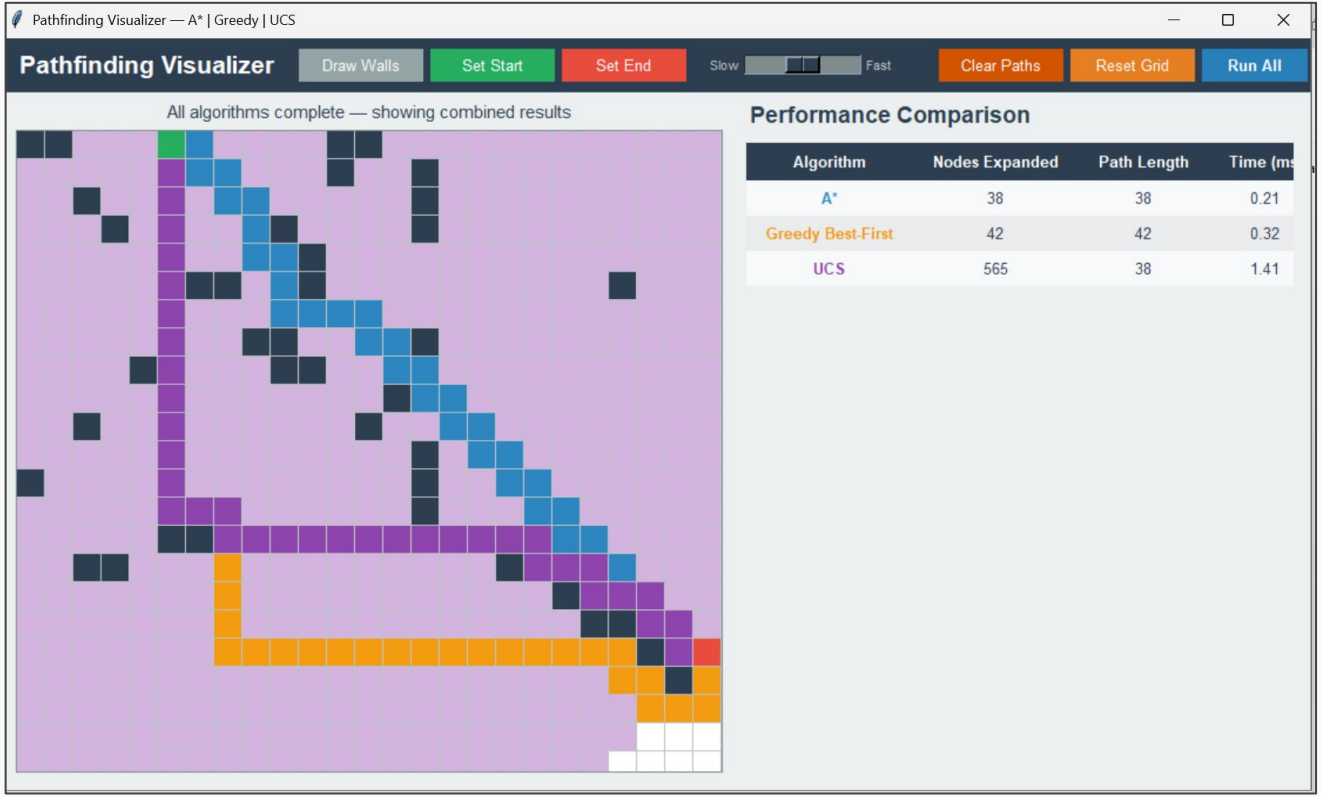
The program was built in Python using Tkinter.

The user can draw walls by clicking and dragging, place the start and end nodes, and then click Run All to execute all 3 algorithms in sequence.

The program animates the explored cells and final path, then shows a results table with nodes expanded, path length, and runtime



Pathway-Finding Visualizer In Progress



Here is an example of the grid interface. The green cell is the start node, the red cell is the goal, and the dark cells are walls. Once the user clicks Run All, the program runs A*, then Greedy, then UCS, and colors their explored areas and paths using different colors.

Conclusion

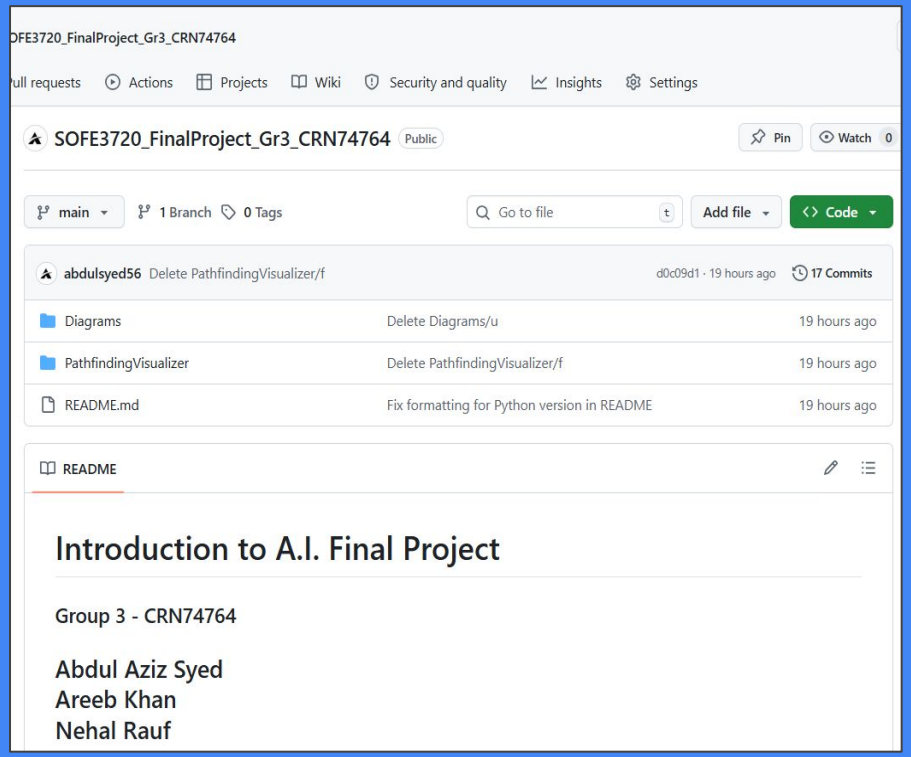
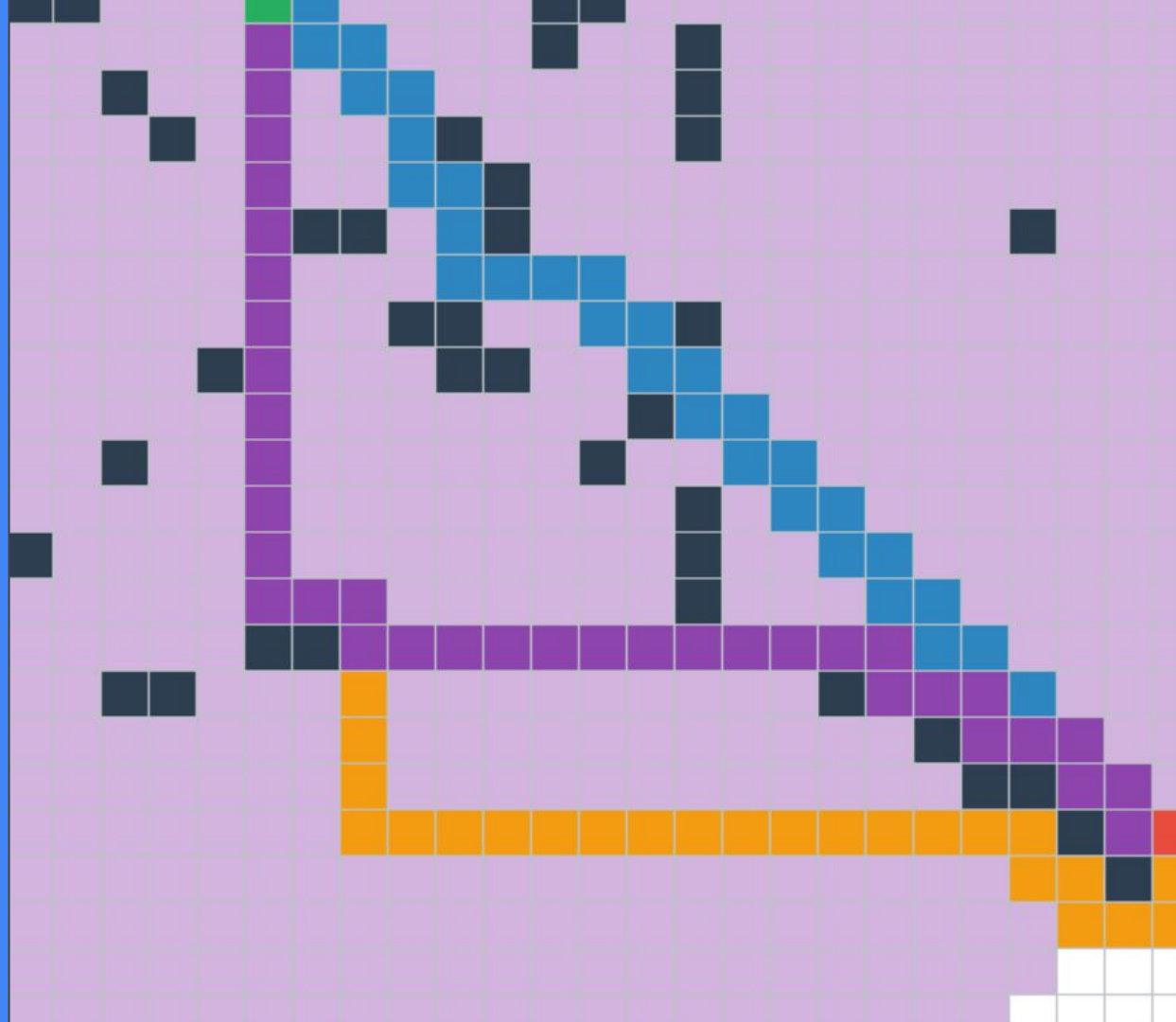
This project was to make a given/chosen AI pathfinding problem into an interactive visual system.

We implemented A*, Greedy Best-First Search, and Uniform Cost Search from scratch, and compared them using both animation and performance metrics.

Overall, the project shows how heuristics, graph search, and cost functions affect algorithm behaviour in a clear and practical way.

Github Link

https://github.com/abdulsyed56/SOFE3720_FinalProject_Gr3_CRN74764.git



The End